CT-159 · DATA STRUCTURES

Doubly & Circular Linked Lists

Lecture 3 — Variations of the Linked List

Presenter: Dr. Muhammad Kamran

SECTION 01

Doubly Linked List

Two links per node — walk the list forward and backward.

SECTION 01 · DOUBLY

01

What is a Doubly Linked List?

A linked list where each node points both to the next AND the previous node.

- A variation of the singly linked list that allows navigation in both directions.
- Forward and backward traversal are both easy and direct.
- The cost: every node stores one extra pointer (prev).

SECTION 01 · DOUBLY

02

Key Points

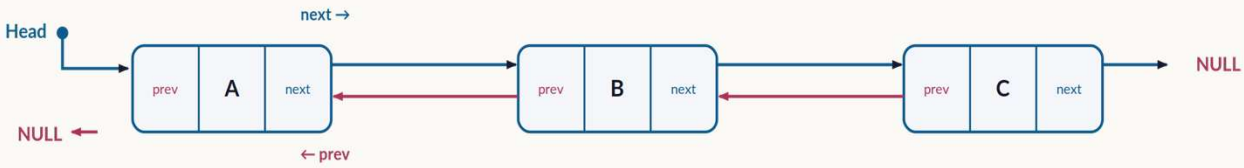
First & Last	The list keeps a reference to both the first and the last node.
Two link fields	Each node has a data field plus next and prev pointers.
next link	Points to the following node in the list.
prev link	Points to the previous node in the list.
NULL ends	The first node's prev and the last node's next are NULL.

SECTION 01 · DOUBLY

03

Doubly Linked List — Structure

next (blue) walks forward; prev (rose) walks backward; both ends terminate in NULL.



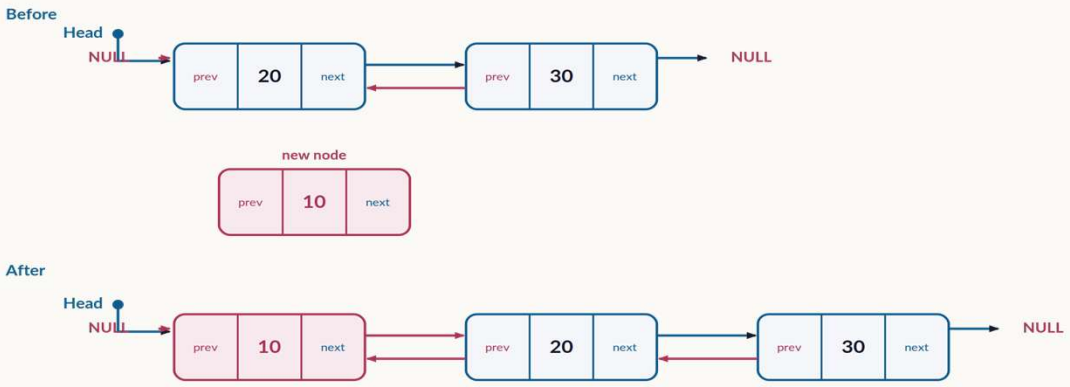
Each node keeps two links: next (forward) and prev (backward) — so the list walks both ways.

SECTION 01 · DOUBLY

04

Doubly — Insertion at Head

Add a node at the front — the new node becomes the head.



Set new.next = head and head.prev = new, then move head to the new node.

SECTION 01 · DOUBLY

05

Insert at Head — Algorithm

Steps

1. Create the new node
2. new.next = head
3. new.prev = NULL
4. If head exists: head.prev = new
5. head = new

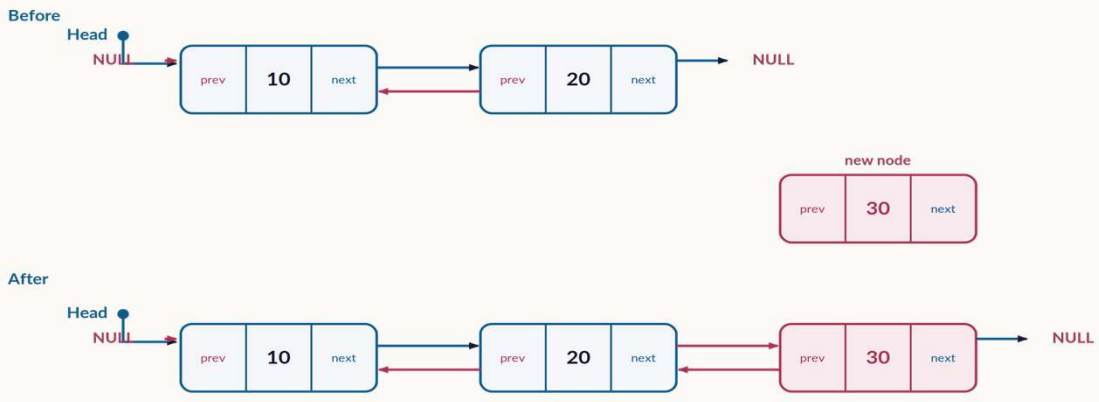
```
// pseudocode
newNode = Node(value)
newNode.next = head
newNode.prev = NULL
if head != NULL:
    head.prev = newNode
head = newNode
```

SECTION 01 · DOUBLY

06

Doubly — Insertion at End

Walk to the last node, then attach the new node behind it.



Walk to the last node; set last.next = new and new.prev = last.

SECTION 01 · DOUBLY

07

Insert at End — Algorithm

Steps

1. If list empty: head = new, return
2. temp = head; walk to the last node
3. last.next = new
4. new.prev = last
5. new.next = NULL

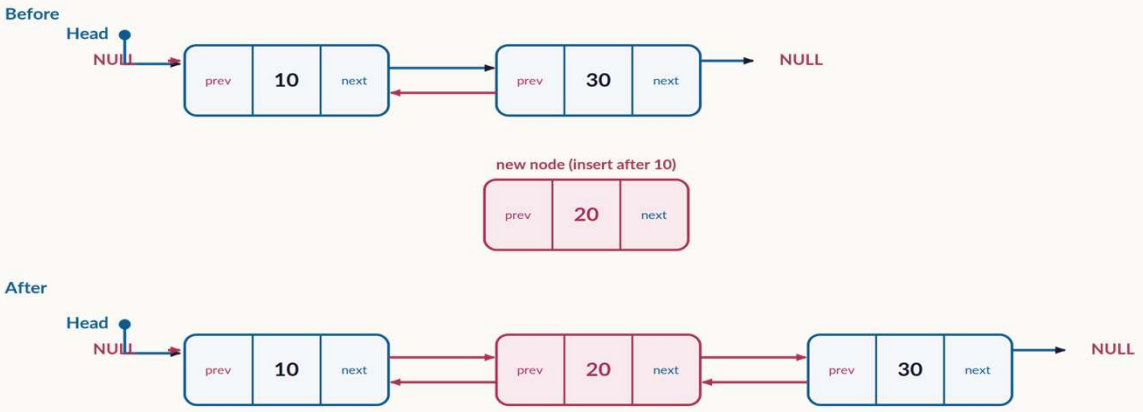
```
// pseudocode
newNode = Node(value)
if head == NULL:
    newNode.prev = NULL
    head = newNode; return
temp = head
while temp.next != NULL:
    temp = temp.next
temp.next = newNode
newNode.prev = temp
newNode.next = NULL
```

SECTION 01 · DOUBLY

08

Doubly — Insertion in the Middle

Insert after a given node — four prev/next links change.



Insert 20 after 10: link 20 between 10 and 30, fixing all four prev/next pointers.

SECTION 01 · DOUBLY

09

Insert in Middle — Algorithm

Steps

1. Reach the node to insert after (curr)
2. new.next = curr.next
3. new.prev = curr
4. If curr.next: curr.next.prev = new
5. curr.next = new

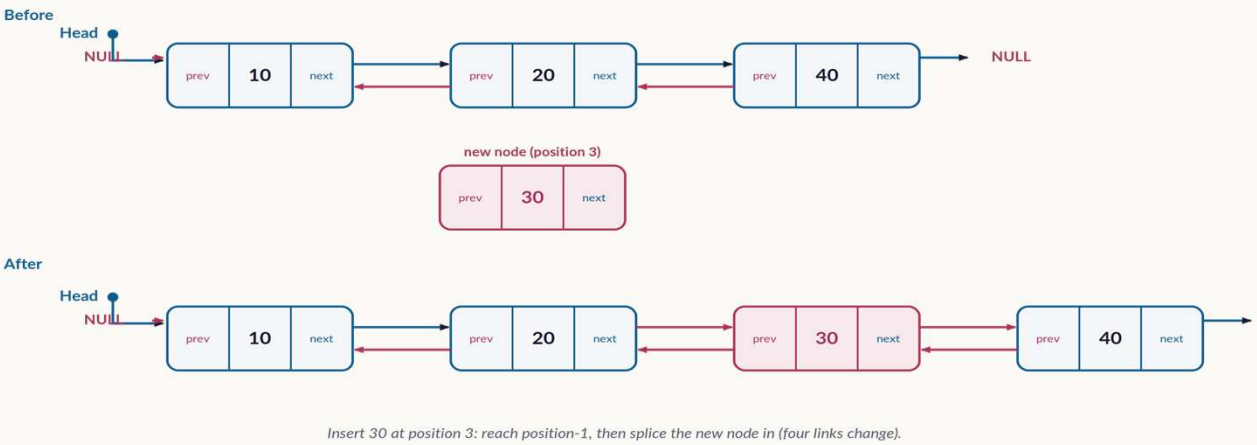
```
// pseudocode
newNode.next = curr.next
newNode.prev = curr
if curr.next != NULL:
    curr.next.prev = newNode
curr.next = newNode
// order matters: fix new's
// links before curr.next
```

SECTION 01 · DOUBLY

10

Doubly — Insertion at a Position

Walk to position−1, then splice the new node in (four links).



Insert 30 at position 3: reach position-1, then splice the new node in (four links change).

SECTION 01 · DOUBLY

11

Insert at Position — Algorithm

Steps

1. If position = 1: insert at head, return
2. curr = head; step to (position − 1)
3. new.next = curr.next
4. new.prev = curr
5. fix curr.next.prev, then curr.next = new

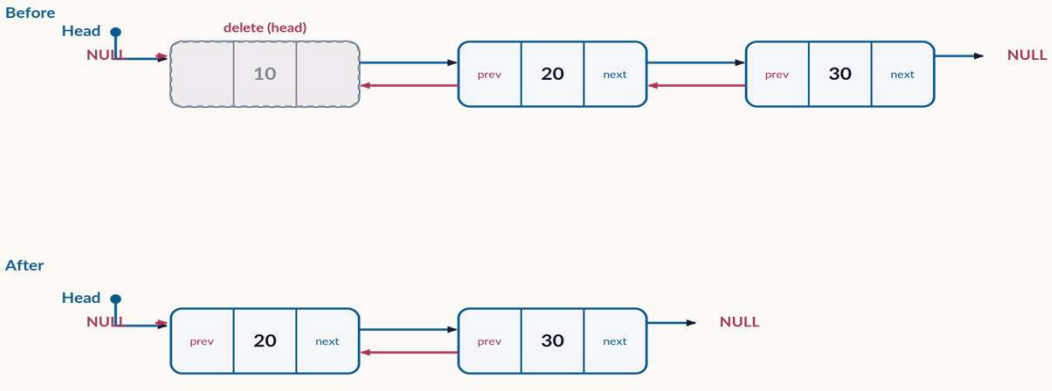
```
// pseudocode
if position == 1:
    insertAtHead(value); return
curr = head
for i in 1 .. position-2:
    curr = curr.next
newNode.next = curr.next
newNode.prev = curr
if curr.next != NULL:
    curr.next.prev = newNode
curr.next = newNode
```

SECTION 01 · DOUBLY

12

Doubly — Deletion from Head

Remove the first node and fix the new head's prev.



Move head to head.next, set new head.prev = NULL, then free the old first node.

SECTION 01 · DOUBLY

13

Delete from Head — Algorithm

Steps

1. If list empty: nothing to delete
2. temp = head
3. head = head.next
4. If head exists: head.prev = NULL
5. free(temp)

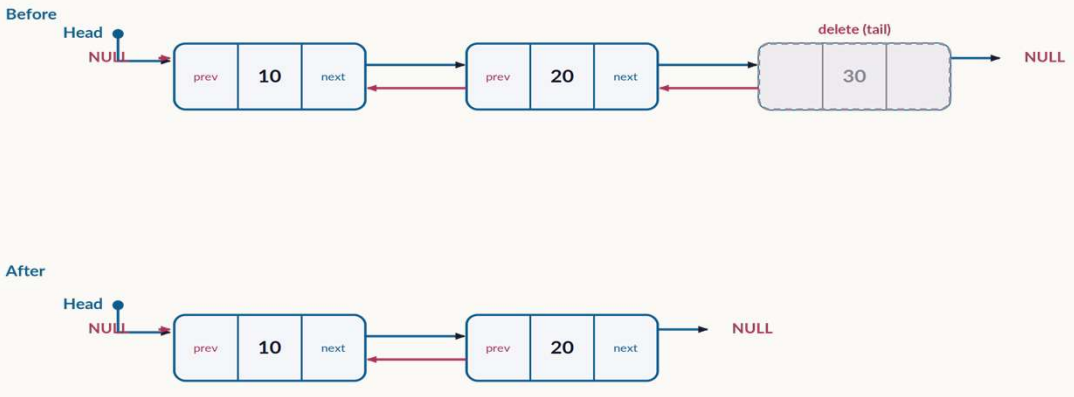
```
// pseudocode
if head == NULL: return
temp = head
head = head.next
if head != NULL:
    head.prev = NULL
free(temp)
```

SECTION 01 · DOUBLY

14

Doubly — Deletion from End

Walk to the last node and detach it.



Reach the last node; set second-last.next = NULL, then free the last node.

SECTION 01 · DOUBLY

15

Delete from End — Algorithm

Steps

1. If empty or single node: handle directly
2. temp = head; walk to the last node
3. temp.prev.next = NULL
4. free(temp)

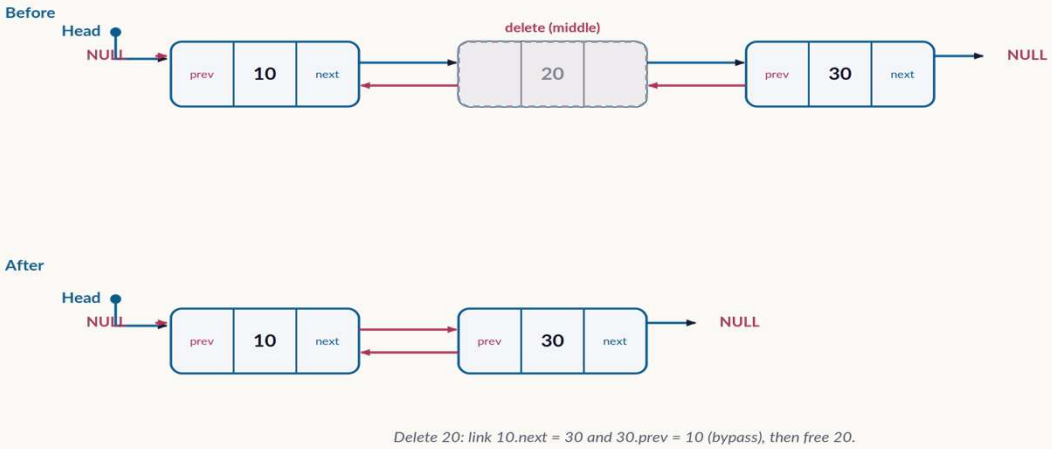
```
// pseudocode
if head == NULL: return
temp = head
while temp.next != NULL:
    temp = temp.next
if temp.prev != NULL:
    temp.prev.next = NULL
else:
    head = NULL      // single node
free(temp)
```

SECTION 01 · DOUBLY

16

Doubly — Deletion from the Middle

Bypass the target on both chains, then free it.



Delete 20: link 10.next = 30 and 30.prev = 10 (bypass), then free 20.

SECTION 01 · DOUBLY

17

Delete from Middle — Algorithm

Steps

1. Reach the node to delete (curr)
2. curr.prev.next = curr.next
3. If curr.next: curr.next.prev = curr.prev
4. free(curr)

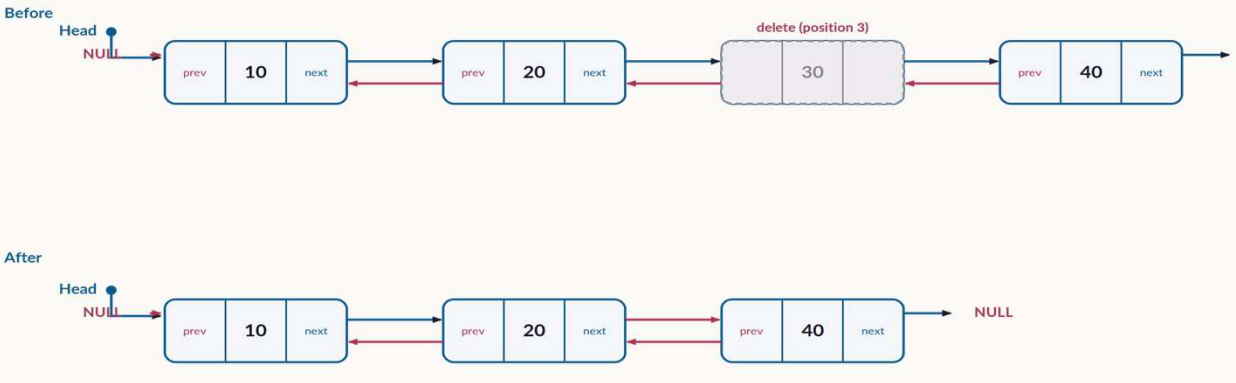
```
// pseudocode
// curr = node to delete
curr.prev.next = curr.next
if curr.next != NULL:
    curr.next.prev = curr.prev
free(curr)
```

SECTION 01 · DOUBLY

18

Doubly — Deletion at a Position

Walk to the position, unlink that node from both neighbours.



Delete at position 3: bypass node 30 by re-linking 20 and 40, then free 30.

SECTION 01 · DOUBLY

Delete at Position — Algorithm

Steps

1. curr = head; step to the position
2. curr.prev.next = curr.next
3. If curr.next: curr.next.prev = curr.prev
4. (handle head/tail edge cases)
5. free(curr)

```
// pseudocode
curr = head
for i in 1 .. position-1:
    curr = curr.next
if curr.prev != NULL:
    curr.prev.next = curr.next
else:
    head = curr.next          // was head
if curr.next != NULL:
    curr.next.prev = curr.prev
free(curr)
```

SECTION 02

Circular Linked List

No NULL end — the last node loops back to the first.

SECTION 02 · CIRCULAR

20

What is a Circular Linked List?

A linked list where the last node points back to the first, forming a loop.

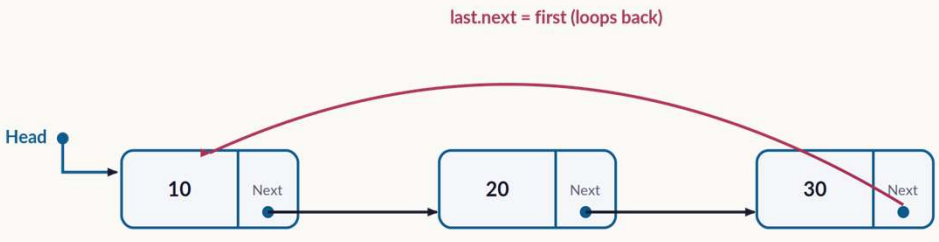
- The first element links to the last, and the last links back to the first.
- Both singly and doubly linked lists can be made circular.
- Following next never hits NULL — it cycles through the list forever.

SECTION 02 · CIRCULAR

21

Singly Linked List as Circular

The last node's next points to the first instead of NULL.



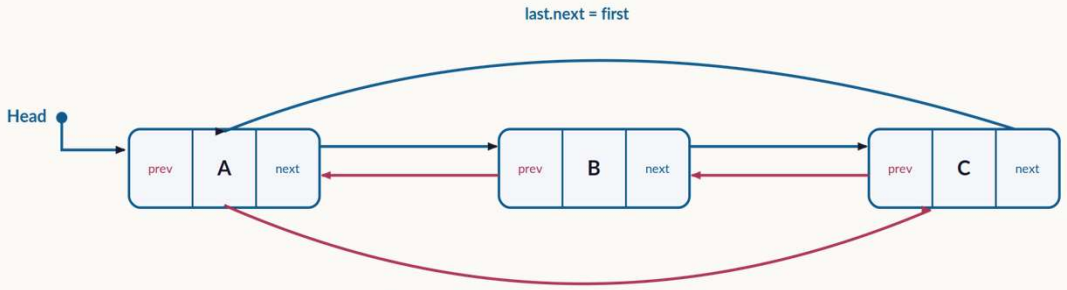
A singly circular list: the last node points to the first — there is no NULL end.

SECTION 02 · CIRCULAR

22

Doubly Linked List as Circular

Both chains close: last.next = first. and first.prev = last.



A doubly circular list closes both the next and prev chains into rings.

SECTION 02 · CIRCULAR

23

Adding a Node at a Position

Steps

1. If position = 1: make new node the head, return
2. Else walk to the node at (position − 1), call it curr
3. Create new node with the data
4. new.next = curr.next
5. curr.next = new

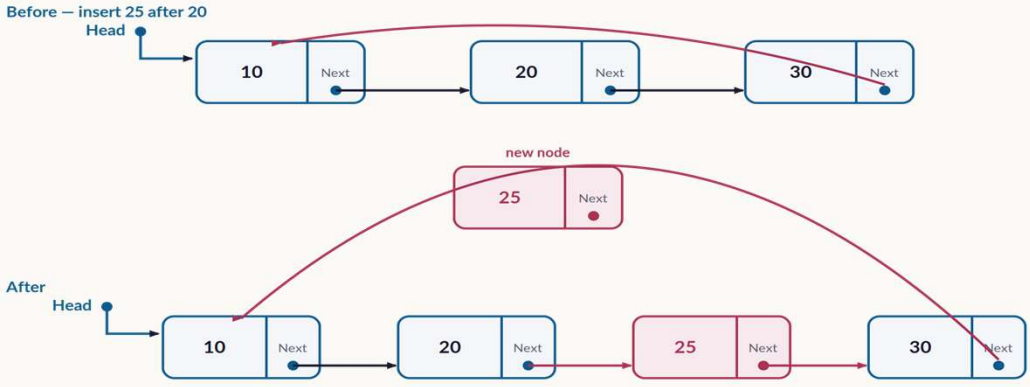
```
// pseudocode
if position == 1:
    newNode.next = head
    head = newNode
    return head
curr = head
for i in 1 .. position-2:
    curr = curr.next
newNode.next = curr.next
curr.next = newNode
```

SECTION 02 · CIRCULAR

24

Singly Circular — Insert in Middle

Inserting mid-list works like a normal singly list; the wrap-around link is untouched.



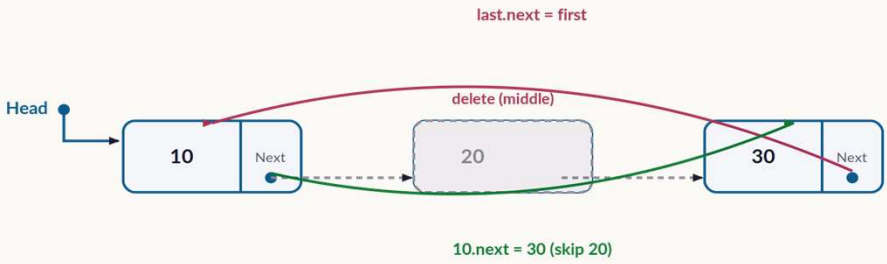
Same as a normal middle insert — the wrap-around link (last → first) is untouched.

SECTION 02 · CIRCULAR

25

Singly Circular — Delete from Middle

Re-link the previous node past the target; the circular loop still holds.



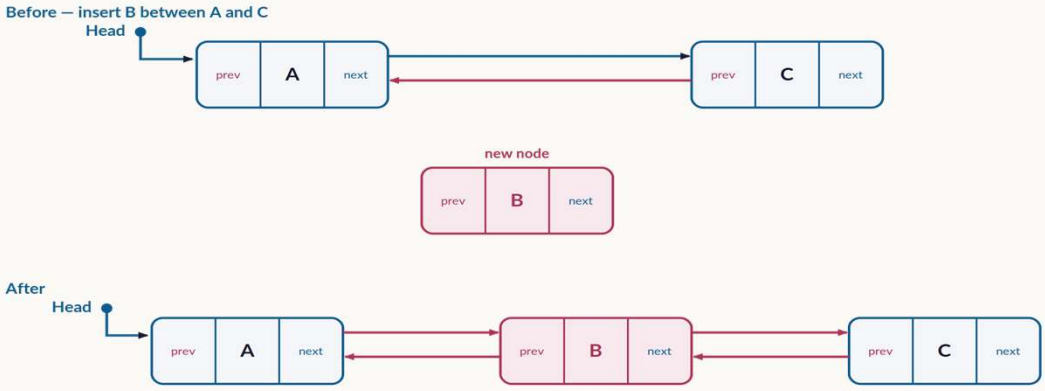
Re-link the previous node past the target; the circular wrap-around still holds.

SECTION 02 · CIRCULAR

26

Doubly Circular — Insert in Middle

Four links change and the ring stays closed on both the next and prev chains.



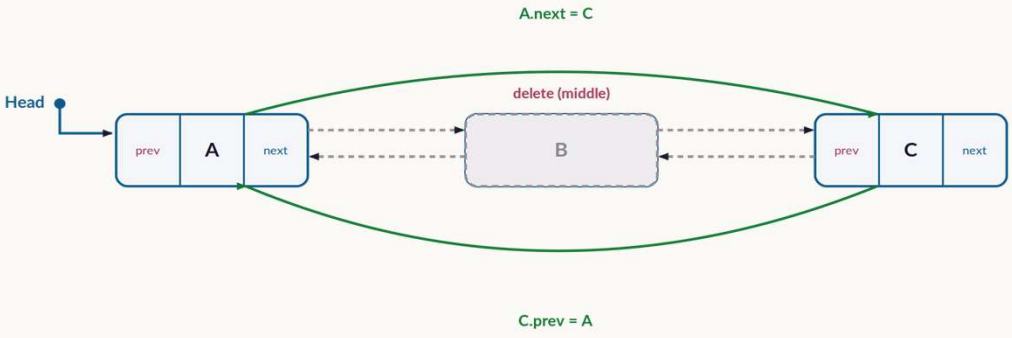
Four links change: B.prev, B.next, A.next and C.prev — the ring stays closed.

SECTION 02 · CIRCULAR

27

Doubly Circular — Delete from Middle

Bypass the node on both chains: A.next = C and C.prev = A, then free it.



Bypass B on both chains: A.next = C and C.prev = A, then free B.

SECTION 02 · CIRCULAR

28

Where Circular Lists Are Used

Round-robin scheduling	CPUs cycle through processes endlessly — a natural loop.
Multiplayer games	Turn order cycles back to the first player.
Circular buffers	Streaming / ring buffers reuse the same slots in a loop.
Music / slideshows	“Repeat all” playlists loop back to the start.

SECTION 03

Practice It Yourself

Pointer problems reward drawing them out — no AI shortcuts.

PRACTICE · LINKED LISTS

P1

Problems to Try

Draw the nodes and every pointer before you code. That's how the intuition sticks.

LeetCode 707	**Design Linked List**	Medium	Implement a linked list with get/insert/delete at index.
LeetCode 430	**Flatten a Multilevel DLL**	Medium	Flatten a doubly linked list that has child pointers.
LeetCode 141	**Linked List Cycle**	Easy	Detect whether a list is circular using two pointers.
LeetCode 142	**Linked List Cycle II**	Medium	Find the node where the cycle begins.
LeetCode 21	**Merge Two Sorted Lists**	Easy	Warm-up: pointer surgery on two lists.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 141 — The Approach

Problem (in short): given a linked list, decide whether it contains a cycle (loops back on itself).

- Use two pointers: slow moves 1 step, fast moves 2 steps each iteration.
- If there is no cycle, fast reaches NULL — return false.
- If there is a cycle, fast eventually laps slow and they meet — return true.
- This is Floyd's "tortoise and hare" — O(n) time, O(1) space.

YOUR TURN

On paper, draw 1 → 2 → 3 → 4 → back to 2. Move slow and fast by hand. On which node do they meet?

PRACTICE · WORKED EXAMPLE

P3

LeetCode 141 — The Code

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;          // 1 step
        fast = fast->next->next;    // 2 steps
        if (slow == fast)
            return true;            // they met -> cycle
    }
    return false;                   // fast hit NULL -> no cycle
}
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

The idea	In a loop, the faster pointer keeps gaining on the slower one until it catches up.
No cycle	fast (or fast.next) reaches NULL, so the loop ends and we return false.
Time	O(n) — fast scans the list at most a constant number of times.
Space	O(1) — just two pointers, no extra set or map needed.

PRACTICE · RESOURCES

P5

Where to Get Help

NeetCode	Floyd's cycle detection, visualised. Search: "NeetCode Linked List Cycle 141"
Abdul Bari	Doubly & circular list fundamentals. Search: "Abdul Bari Doubly Linked List"
William Fiset	Full linked-list playlist. Search: "William Fiset Doubly Linked List"

YOUR TURN

Course rule: try each problem for 20 minutes and draw the pointers before watching anything — and never paste it into an AI.

LECTURE 3 — RECAP

What we covered

- Doubly lists: next + prev, two-way traversal
- Insertion updates four pointers
- Circular lists loop back — no NULL end
- Singly & doubly can both be circular
- Uses: scheduling, ring buffers, playlists

Next: Stacks & Recursion